Couchbase

Couchbase Under the Hood



Contents

INTRODUCTION **3**

Essential NoSQL requirements and features 3
The original multi-model NoSQL database 6
Core performance design principles 6
Couchbase Capella DBaaS 8
Capella Free Tier 8

JSON DATA MODEL AND ACCESS METHODS **8**

JSON document data model 9
Document access methods 11
Key, values, and sub-documents 12
Key organizing concepts for documents 13
Cluster design concepts 14

CORE DATABASE SERVICES **15**

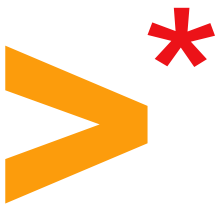
Data Service and Key/Value Engine 15
Key-value data access 18
Data Storage Engines: Magma and Couchstore 19
Query service 19
Capella iQ and developer tools 22
Index service 22
Full-Text Search 24
Geospatial search 25
Vector search 26
Eventing service 27

DISTRIBUTED FOUNDATION **28**

Node-level architecture 28
Cluster architecture 29
Client connectivity 31
Data transport via Database Change Protocol (DCP) 32
Multi-Dimensional Scaling (MDS) 32
Independent scaling model 33
Data distribution 34
Rebalancing the cluster 35
High availability 36
Security 43
Closing 45

RESOURCES **46**

INTRODUCTION



Couchbase Enterprise is a modern distributed, multi-model NoSQL database. Couchbase's core architecture supports a flexible JSON data model at its foundational exchange format, includes a high-performance key-value database for persistence, and uses familiar relational and multi-model data access services to supply data to operational and AI applications. The Couchbase database platform advantages include fast in-memory performance, easy scalability, real-time vector search at billion scale, always-on 24x365 availability, advanced security, and affordable cloud deployment options. Couchbase can be deployed as a fully-managed database-as-a-service called Couchbase Capella, and also offers Kubernetes-managed containerized cluster deployments with its CNCF-based Couchbase Autonomous Operator packaging. Couchbase also supports self-managed installations of its Community and Enterprise edition binary packages.

In its multipurpose design, Couchbase supports multiple data access methods within a dynamic data containment structure, on top of a flexible JSON document data format. Couchbase consolidates multiple data access layers and engines into a single platform that would otherwise require multiple single-purpose databases to work together. This "polyglot persistence" design architecture was introduced in the early 2000's so that RDBMS and specialty NoSQL databases could coexist in supplying data to applications. This data architecture has become the de-facto standard practice offered by the hyperscalers since then. Couchbase provides the performance of a key-value powered caching layer, the flexibility of a JSON document-based dynamic source of truth, and the reliability of a relational database system of record. Couchbase eliminates the need to manage strict data models and consistency between multiple systems, learn different languages and APIs, and manage independent technologies hosting redundant data.

This paper describes how the internal components of the Couchbase Enterprise database server operate with one another. It assumes you have a basic understanding of Couchbase and are looking for a deeper technical understanding of how things operate beneath the surface. Additional whitepapers exist with more details related to the **Couchbase Data Platform for AI** as a whole and for **Couchbase Mobile** edge, IoT and mobile apps.

Essential NoSQL requirements and features

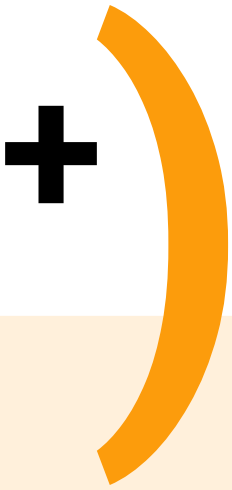
NoSQL databases evolved beyond enterprise relational databases to address performance and flexibility deficiencies made evident as applications became more sophisticated and "Big Data" became an industry-standard buzzword. Relational databases tend to operate primarily as systems of record, maintaining transactional data in a highly consistent structured manner. But several architectural principles (e.g. normalization of objects, adherence to fixed schema and data typing, single node transactional design, and two-phase commits) have made them difficult to modify after deployment and challenging to scale larger distributed workloads while simultaneously delivering responsive, highly available applications.

Pragmatic business necessity for more advanced technical requirements have pushed multi-model NoSQL databases to the forefront. The business demands for high performance, application-driven flexibility over the makeup of its data, distributed processing and mobility, and the overarching aspiration to lower operational costs and escape vendor lock-in are key drivers as to why organizations seek out cloud-native NoSQL systems. These modern requirements have driven Couchbase's development from inception:

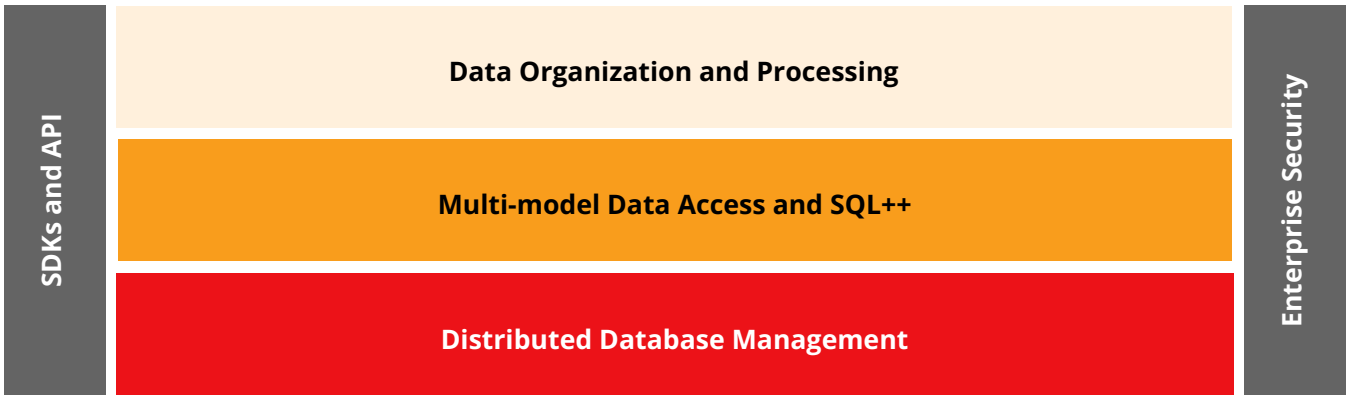
- **Ensure high-performance and availability**
- **Provide data model and data access flexibility**
- **Support distributed cluster networks and mobility**
- **Provide incredible value and low TCO**

Couchbase Enterprise has been focused on setting a high standard in each of these areas. The result is a robust and accessible database platform with exceptional performance at scale, multi-model flexibility, and SQL familiarity delivered in both self-managed and fully-managed cloud-based product lines. Because of this, Couchbase has become a modern, multipurpose NoSQL database. Learn more about how Couchbase delivers on these critical database requirements in the following table and later in the text of this paper.

- **Power Your Performance** – Expect peak performance from your digital experiences, even at peak demand
- **Accelerate Your Innovation** – Get to market faster and stay one step ahead of competitors with a unified data platform for AI
- **Simplify Your Operations** – Cut complexity and drive visibility by consolidating your legacy infrastructure and services
- **Control Your Costs** – Optimize your infrastructure spending with a unified database that significantly reduces your TCO



Power Your Performance	Accelerate Your Innovation	Simplify Your Operations	Control Your Costs
• Memory-and-network-centric architecture, with an integrated cache delivering high throughput and sub-millisecond latency. • Asynchronous clusters. • Global deployment with low-write latency using active-active cluster availability and cross-cloud replication. • Microservices architecture with built-in auto-sharding, replication, and failover.	• Flexible JSON data model supports dynamic applications. Make schema changes without downtime. • Extract value using a broad set of multi-model data access capabilities (full-text search, vector search, data streaming, change-data-capture, and python and JavaScript-based User-Defined Functions). • Efficient vector search with 3 indexing options. • SDKs for 12+ languages, including mobile.	• Fully-managed Database-as-a-service without cloud vendor lock-in. • Self-managed Kubernetes-based cloud native database automation with Autonomous Operator. • Infrastructure agnostic support across local, virtual machines, clouds, and containerized environments. • Always-on, fault-tolerant design. • Includes foundational RDBMS concepts like schema, ACID transactions and User-defined functions.	• High performance at scale means teams can do more with less infrastructure, lowering TCO. • Isolated and independent scaling of workloads, with no downtime for upgrades or code changes. • Infrastructure agnostic support across local, virtual machines, clouds, and containerized environments.





The original multi-model NoSQL database

Couchbase was originally founded through the merger of two open source database companies, CouchOne and Membase. CouchOne employed developers of Apache CouchDB, an original, highly reliable, JSON document database, while Membase employed developers of memcached, a popular high-performance, memory-first, key-value database. The merger of these teams led to the design of Couchbase, a reliable, scalable, fast in-memory, key-value database with document-based access and storage. In this model, document identification "keys" present"value" data as a JSON document. Couchbase was the first of its kind, a dual-model access database, setting the standard for advancing consolidation of single-model, NoSQL datastores with proprietary persistence structures. Couchbase further distanced itself from its original genealogy by adding support for SQL++ (formerly known as N1QL) as its primary query language. Today, multi-model convergence continues to grow in order to address the variety of functional and complexity demands from modern applications.

Unfortunately, many people still confuse Couchbase with CouchDB even though they have evolved along their own diverging paths, and no longer resemble each other whatsoever.

COUCHBASE IS AN OPEN SOURCE DATABASE COMPANY

Couchbase favors and supports the open source development model. The source code to the Community Editions of the Couchbase database and its mobile product line is available for non-commercial use under the Business Software License (BSL 1.1), which converts to the permissive Apache 2.0 license after four years. Software development kits (SDK's) for more than a dozen applications and mobile programming languages are available as Apache 2.0 open source. Couchbase also maintains a robust GitHub library of open source projects at couchbaselabs.

These principles of speed, flexibility, familiarity, and affordability have been built in the very core of the database engine to ensure low latency and reliable, yet easy to manage, replication. Around this core are a set of data access services that run and scale independently of each other. These are delivered through a unified programming API, established security capabilities, and external technology integrations, and made available through fully managed and self-managed offerings including Couchbase Capella, a fully-hosed database-as-a-service, and through the Kubernetes-based, Cloud Native Database Automation product line for self-managed deployments.

Core performance design principles

To effectively deliver the above features, three guiding principles have been followed when developing Couchbase: memory and network-centric architecture, workload isolation, and an asynchronous approach to everything.

MEMORY AND NETWORK-CENTRIC ARCHITECTURE FOR SPEED AND LOW LATENCY

- The most used data and indexes are transparently cached in memory for fast reads.
- Writes are performed in memory and replicated or persisted synchronously or asynchronously. Using transaction guarantees ensures consistency, but may introduce lags in performance.
- Internal Database Change Protocol (DCP) streams data mutations from memory to memory at network speeds to support replication, indexing, and mobile synchronization.



MULTI-MODEL DATA ACCESS BLENDING JSON FLEXIBILITY WITH KEY/VALUE SPEED

Couchbase is a pioneer in offering multiple data access methods to gain read and update access to its foundational JSON and Key/Value storage structures. This type of NoSQL database is referred to as "multi-model" because many NoSQL systems have only one access method, which is bound to their physical storage design structures on disk to minimize access latency.

Couchbase's original access methods are:

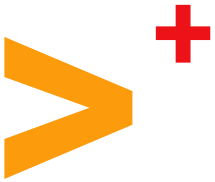
- Key/Value, derived from Memcached-based design and
- JSON, the de facto standard format for document databases like CouchDB, whose authors based the Couchbase design on their earlier work. While many developers may confuse the two, Couchbase and CouchDB are not the same; Couchbase should be considered as the unique offspring of the developers of CouchDB and Memcached, sharing DNA from both.
- As it evolved, Couchbase has added multiple data access models, including a SQL++ query service, a Full-Text Search service, Vector Search, an Eventing service, and a Backup service. In the Couchbase design, each of these access models can simultaneously utilize the cluster's data.

WORKLOAD ISOLATION

- All databases perform different tasks in support of an application. These include persisting, indexing, querying, aggregating, and searching data. Each of these workloads has slightly different performance and resource requirements.
- Multi-Dimensional Scaling (MDS) isolates these workloads from one another at both a process and a node level.
- MDS allows these workloads to be scaled independently from one another and their resources to be optimized as necessary.
- MDS allows the database to be performance-matched to the performance needs of the application, and the database to its available infrastructure. For cloud deployments, it is advantageous from a cost perspective to "red-line" infrastructure instances before adding them and to avoid idle and under-utilized node instances.
- Couchbase manages the topology, process management, statistics gathering, high availability, and data movement between these services transparently.

ASYNCHRONOUS APPROACH TO EVERYTHING

- Traditional databases increase latency and block application operations while running synchronous operations, for example, persisting data to disk or maintaining indexes.
- Couchbase allows write operations to happen at memory and network speeds while asynchronously processing replication, persistence, and index management.
- Spikes in write operations don't block read or query operations, while background processes will persist data as fast as possible without slowing down the rest of the system.
- ACID transactions are available to the developer to ensure durability and consistency while data is in flight. Multiple transaction options are available, allowing the developer to decide when and where to increase latency in exchange for durability and consistency of transactions. Somewhat higher latency can be anticipated as multi-document and cross-collection transactions are implemented.



Couchbase Capella DBaaS

Capella is the fastest, easiest, and most affordable way to start with Couchbase. As a fully managed DBaaS, it automates setup, configuration, replication, and ongoing operations so you can focus on development and improve your time to market. And Capella puts industry-leading performance, flexibility, and cloud scalability at your fingertips.

Capella delivers flexibility across your use cases with built-in multi-model capabilities. And its memory-first architecture drives millisecond data response at scale, resulting in the best price-performance of any fully managed document database. Capella provides guaranteed global reliability across 85+ regions across Amazon Web Services, Microsoft Azure, and Google Cloud platforms.

Capella Free Tier

Capella's free tier gives users access to the core capabilities for initial exploration and development. By offering a fully managed database environment at no cost, we're enabling developers across organizations to experiment and build without the financial constraints that can stifle creativity. Now, anyone with an idea has access to our powerful database platform, and can start building immediately.

JSON DATA MODEL AND ACCESS METHODS

This section outlines the foundational JSON data model handling in Couchbase, then introduces the multiple ways to access that data. These methods include basic key-value operations, SQL++ querying, full-text and vector searching, and server-side eventing.

JSON document data model

The JSON data model supports basic and complex data types: numbers, strings, nested objects, and arrays. JSON provides rapid serialization and deserialization, is native to JavaScript, and is the most common REST API data format. Consequently, JSON is extremely convenient for web application programming.

Couchbase stores data as individual documents, consisting of a key and a value. When the value is JSON-formatted, Couchbase provides rich access capabilities; when not, the document is stored as a binary BLOB and has more limited access characteristics.

A document often represents a single instance of an application object (or nested objects). It can also be considered analogous to a row in a relational table, with the document attributes acting similar to a column. Couchbase provides greater flexibility than the rigid schemas of relational databases, allowing JSON documents with varied schemas and nested structures. Developers may express many-to-many relationships without requiring a reference or junction table. Subcomponents of documents can be accessed and updated directly as well, and multiple document schemas can be aggregated together into a virtual table with a single query.

JSON DOCUMENT FLEXIBILITY

In the Couchbase document model, a schema is the result of an application's structuring of its documents and their containment structures such as Buckets, Scopes and Collections. Schemas are defined by application developers and managed by applications. This is in contrast to the relational model, where the database (and the database administrator) manages the schema. Couchbase created the Bucket-Scope-Collection-Document organizational hierarchy, explained below, so as to allow maximum flexibility in defining application data meta models. Notice how easily traditional RDBMS constructs map to those of Couchbase:

Couchbase	RDBMS
Bucket	Database
Scopes	Schema
Collections	Tables
Documents	Rows
Key/Value pairs	Cells



A single JSON document's structure offers even more flexibility for the developer beyond the dynamic nature of Scopes and Collections. A JSON document's structure consists of its inner arrangement of attribute-value pairs. For example, both of the following JSON document examples are valid data models that Couchbase can manage and query. How the documents are designed or updated over time is up to the application developer – normalized or denormalized, or a hybrid, depending on the needs and evolution of the application. Using JSON, the developer can avoid lengthy schema design, testing and deployment cycles of traditional RDBMS-based systems.

Normalized – 4 Documents

```
invoice1:
{
  "BillTo": "Lynn Hess",
  "InvoiceDate": "2018-01-15",
  "InvoiceNum": "ABC123",
  "ShipTo": "H. Trisler, 41
  Oak Drive"
}
invoice1:item1:
{
  "InvoiceId": "1",
  "Price": "100",
  "Product": "Brake Pad",
  "Quantity": "24"
}
invoice1: item2
{
  "InvoiceId": "1",
  "Price": "10",
  "Product": "Rotor",
  "Quantity": "5"
}
invoice1: item3
{
  "InvoiceId": "1",
  "Price": "20",
  "Product": "Tire",
  "Quantity": "2"
}
```

Denormalized – 1 Document

```
invoice1:
{
  "BillTo": "Lynn Hess",
  "InvoiceDate": "2018-01-15",
  "InvoiceNum": "ABC123",
  "ShipTo": "H. Trisler, 41 Oak Drive",
  "Items": [
  { "Price": "100", "Product": "Brake Pad", "Quantity": "24"
  },
  { "Price": "10", "Product": "Rotor", "Quantity": "5" }
  { "Price": "20", "Product": "Tire", "Quantity": "2" }
 ]
}
```

Couchbase does not enforce uniformity: document structures can vary, even across multiple documents where each contains a type attribute with a common value. This allows differences between objects to be represented efficiently. It also allows a schema to progressively evolve for an application, as required – properties and structures can be added to the document without other documents needing to be updated in the same way. This allows applications to change their behavior without having to overhaul all the source data or take applications offline to make a basic change.



Document access methods

Managing JSON data is at the core of Couchbase's document database capabilities, but there are several ways for applications to access the data. Each of these methods is described in further detail later in this paper, but the following provides a basic explanation and coding example of using it.

Access Method	Description	Example
Key-value	An application provides a document ID (the key), Couchbase returns the associated JSON or binary object. The inverse occurs with a write or update request.	**Java:** `JsonDocument myAirline = collection.get("airline_5209");`
Query and Analytics	SQL-based query syntax to interact with JSON data, similiar to relational databased, returns matching JSON results. Comprehensive DML, DQL and DDL syntax supports nested data and non-uniform schema.	**Python:** `cluster.query(( """SELECT fname, lname, age FROM default WHERE age > $age """, age=22 )`
Full-text Search	Using text analyzers with tokenization and language awareness, a search is made for a variety of field and Boolean matching functions. Search returns document IDs, relevance scoring, and optional context data.	**.NET:** `await cluster.SearchQueryAsync( "fts-index", new QueryStringQuery("sushi"), options => { options.Limit(10); } );`
Vector Search	Documents with embedded vectors are indexed and queried to find similar or semantically related content, supporting pure vector similarity searches, filtered searches using scalar fields, and hybrid searches combined with full-text or geospatial criteria.	**/* Hyperscale index */** `SELECT b.color, b.colorvect_l2, b.brightness from 'rgb' AS b ORDER BY APPROX_VECTOR_DISTANCE(b.colorvect_l2,[128,128,128],"L2") LIMIT 5;`
Eventing	Custom JavaScript functions are executed within the database as data changes or based on timers. Support for accessing and updating data, writing out to a log or calling out to an external system.	**JavaScript:** `function OnUpdate(doc, meta){ log('document', doc); doc["ip_num_start"]= ip_trim_func(doc["ip_start"]); tgt[meta.id]=doc; }`

Key, values, and sub-documents

Keys and values are fundamental parts of JSON documents and have some limits that are important to understand.

KEYS

Each value is identified by a unique key, or ID, defined by the user or application when the item is originally created. The key is immutable: once the item is saved, the key cannot be changed.

Each key must be a UTF-8 string with no spaces. Special characters, such as (, %, /, ", and _, are acceptable, but the key may be no longer than 250 bytes and must be unique within its bucket.

VALUES

The maximum size of a value is 20 MB. Each document consists of one or more attributes, each of which has its own value. An attribute's value can be a basic type, such as a number, string, or Boolean; or a complex type, such as an embedded document or an array.



JSON documents can be parsed, indexed, and queried by other Couchbase services. A value can also be any form of binary, though it won't be parsed, indexed, or queried.

SUB-DOCUMENTS

A sub-document is an inner component of a JSON document. The sub-document API uses a path syntax to specify attributes and array positions to read/write. This makes it unnecessary to transfer entire documents over the network when only partial modifications are required.

Document key: *user200*

```
{
 "age": 21,
 "fav_drinks": {
  "soda": [ "fizzy", "lemon" ]
  }
 "addresses": [
  { "street": "pine" },
  { "street": "maple" }
 ]
}
```

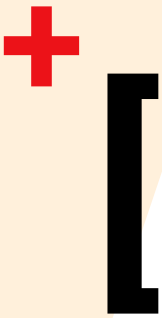
Path Example	Result
age – a numeric value	21
fav_drinks.soda – an array of strings	fizzy, lemon
fav_drinks.soda[0] – first string in array	fizzy
addresses[1].street – string value in second part of array	maple

Key organizing concepts for documents

FLEXIBLE, DYNAMIC DATA CONTAINMENT MODEL

Couchbase offers a flexible, multi-level data containment and organization structure to organize documents, which helps optimize cluster performance and facilitate horizontal scaling. This data containment model consists of four levels: Buckets, Scopes, Collections and Documents, and maps easily to familiar RDBMS constructs of databases, schema, tables and rows. The model structure is further explained below:

- **Buckets:** The topmost container in Couchbase is the Bucket. One or many Buckets can be defined and assigned to a Couchbase cluster. A Bucket is the logical equivalent of a database in relational systems.
- **vBuckets**: Virtual Buckets are how Couchbase creates and uses its data location map, by segmenting a bucket into 1024 vBuckets. vBuckets are distributed across nodes in a cluster and replicated for availability and redundancy.
- **Ephemeral buckets:** Ephemeral buckets exist in memory only. This allows the database to support applications where data is processed, but not permanently persisted.
- **Scope:** Scopes are an intermediate data organization structure similar to a relational database schema. Scopes are defined by the collections of documents that they contain or can access.
- **Collections:** Collections are categorical or logically organized groups of documents. The premise of Collections is to behave as traditional table structures. Most group-oriented access activities are processed at the Collection-level, to minimize full-database operations, simplify replication logic and streamline indexing options.



BUCKETS

Buckets hold scopes, collections, and JSON documents – these are the primary organizing structures in Couchbase. Applications connect to a specific bucket that pertains to their application scope. Applications query data by inquiring about documents within collections inside that scope. Memory quotas are managed on a per-bucket and per-service basis. Security roles are applied to users with various bucket-level, scope-level, collection-level, and document-level constraints.

In addition to standard Couchbase buckets, there are two specialized bucket types useful for different use cases.

Ephemeral buckets do not persist data but allow highly consistent in-memory performance, without disk-based fluctuations. This delivers faster node rebalances and restarts.

Memcached buckets also do not persist data. It is a legacy bucket type designed to be used alongside other database platforms, specifically for in-memory distributed caching. Memcached buckets lack most of the core benefits of Couchbase buckets, including compression.

VBUCKETS

vBuckets are shards or partitions of data (and replicas) that are automatically distributed across nodes. Couchbase automatically segments buckets into 1024 vBuckets. They are managed internally by Couchbase and not interacted with directly by the user. The Couchbase SDK automatically distributes data and workloads across vBuckets using this information as the data location map for the application.

SCOPES

Scopes are a mid-level organizing structure in Couchbase that simplifies support for isolating data and access to that data for the purposes of supporting concepts such as multi-tenancy or separating departmental access to sensitive data.

COLLECTIONS

Collections offer a similar construct to relational tables, in that they contain documents that are alike. Collections-level processing enables Couchbase to isolate operations on smaller data sets, hence improving performance. Indexes are built and managed at the collection level, for example, making queries not only fast but portable. Data sharding and rebalancing is also a collection-level operation. Collections, like tables, can be joined and filtered.

DOCUMENTS

Documents are the foundational construct of Couchbase and conform to JSON structural standards.

Cluster design concepts

NODES

Couchbase nodes are physical or virtual machines that host single instances of Couchbase Server. Multiple instances of Couchbase Server cannot be installed on a node.

CLUSTERS

A cluster consists of one or more nodes running Couchbase Server. Nodes can be added or removed from a cluster. Replication of data occurs between nodes and cross datacenter replication occurs between different clusters that are geographically distributed.

SERVICES

The core of Couchbase is the Data Service that feeds and supports all the other systems and data access methods. There are multiple services, however, that offer different types of data access or processing including, Query, Indexing, Backup, Full Text Search, Analytics and Eventing. A service is an isolated set of processes dedicated to particular tasks. For example, indexing, search, or querying are each managed as separate services. One or more services can be run on one or more nodes as needed.

CORE DATABASE SERVICES



Couchbase implements the above data access methods through a set of dedicated services, with the Data Service at its center. Each service has its own resource quotas and, where applicable, related indexing and inter-node communication capabilities. This provides several very flexible methods to scale services when needed – not just scaling up to larger machines or scaling out to more nodes. Couchbase provides both options, as well as the ability to scale specific services independently from one another. Multi-dimensional scaling is covered in a later section but is the foundation for Couchbase workload isolation and scaling capabilities.

This is different from other platforms where a monolithic set of services are installed on every node in a cluster. Instead, Couchbase uses a core data capability that then feeds all the other services. A shared-nothing architecture allows developer control over workload isolation. Small-scale environments can share the same workloads across one or more nodes, while higher scale and performance can be achieved with dedicated nodes to handle specific workloads – the ultimate in scale-out flexibility. The cluster can be scaled in or out and its service topology changed on-demand with zero interruption or change to the application.

Applications communicate directly with each service through a common SDK that is always aware of the topology of the cluster and how services are configured. Developers do not have to know anything about how the services and nodes are configured, the SDK gets all the information it needs from the cluster manager. The same application can be deployed against a cluster of any size or configuration without changing its behavior. Having this knowledge built into the SDK results in reduced latency (direct application to database access) and less complexity (no proxy or router components), better performance, and simplified auto-scaling.

The core data service handles all the document mutations and streams them to all the related services described below. The remainder of this section walks through each service and describes how they work on a node, in a cluster, and with each other.

Later in this paper, the Distributed Foundation section will discuss inter-node connectivity, data flow, cluster topology, and data streaming.

Data Service and Key/Value Engine

The Data Service is the foundation for storing data in Couchbase Server must run on at least one node of every cluster – it is responsible for caching, persisting, and serving data to applications and other services within the cluster. The principal component of the data service architecture is the key-value management system known simply as the KV Engine.

The KV engine is composed of a multi-threaded, append-only storage layer on disk with a tightly integrated managed object cache. The cache provides consistent low latency for individual document read and write operations and streams documents to other services via DCP.

Each node running the data service has its own KV engine process and is responsible for persisting and caching a portion of the overall dataset (both active and replica).

MANAGED OBJECT CACHE

The managed object cache of each node hashes the document into an in-memory hash table based upon the document ID (key). The hash table stores the key, the value, and some metadata associated with each document. Since the hash table is in memory and lookups are fast, it offers a quick way of detecting whether the document exists in memory or not.

The cache is both read-through and write-through: if a document being read is not in the cache, it is fetched from disk, and write operations are written to disk after being first stored in memory.

Disk fetch requests are batched to the underlying storage engine, and corresponding entries in the hash table are filled. After the disk fetch is complete, pending client connections are notified of the asynchronous I/O completion so that they can complete the read operation.

DOCUMENT EXPIRATION

Documents may also be set to expire using a time to live (TTL) setting. By default, all documents have a TTL of 0, meaning the document will be kept indefinitely. When you add, set, or replace a document, you can specify a custom TTL, at which time the document becomes unavailable and is marked for deletion (tombstone) to be cleaned up later.

MEMORY MANAGEMENT

To keep memory usage of the cache under control, Couchbase employs a background task called the item pager. This pager runs periodically (to clean up expired documents) as well as being triggered based on high and low watermarks of memory usage. This high watermark is based on the memory quota for a given bucket and can be changed at runtime. When the high watermark is reached, the item pager scans the hash table, ejecting eligible (persisted) items that are not recently used (NRU). It repeats this process until memory usage falls below the low watermark.



COMPRESSION

End-to-end data document compression is available across all features of the database using the open source Snappy library. Client capabilities and the compression mode are configured for each bucket, determining how compression will run.

Data can optionally be compressed by the client (SDK) prior to writing into a bucket, within the memory of the bucket and on disk. It is also compressed between nodes of the cluster and to remote clusters.



Compression modes

Data is always compressed on disk, but each client and bucket can control whether it is also compressed on the wire and/or in memory. The SDKs communicate whether they will be sending or requesting compressed documents, and the compression mode of the bucket determines what happens within the database. The modes are as follows:

Off – Documents are actively decompressed before storing in memory. Clients receive the documents uncompressed.

Passive – Documents are stored in memory both compressed and uncompressed in memory, depending on how the client has sent them. Clients receive compressed documents if they are able and uncompressed if they are not.

Active – Documents are actively compressed on the server, regardless of how they were received. Clients receive compressed data whenever it is supported by the client, even if it was originally sent uncompressed data.

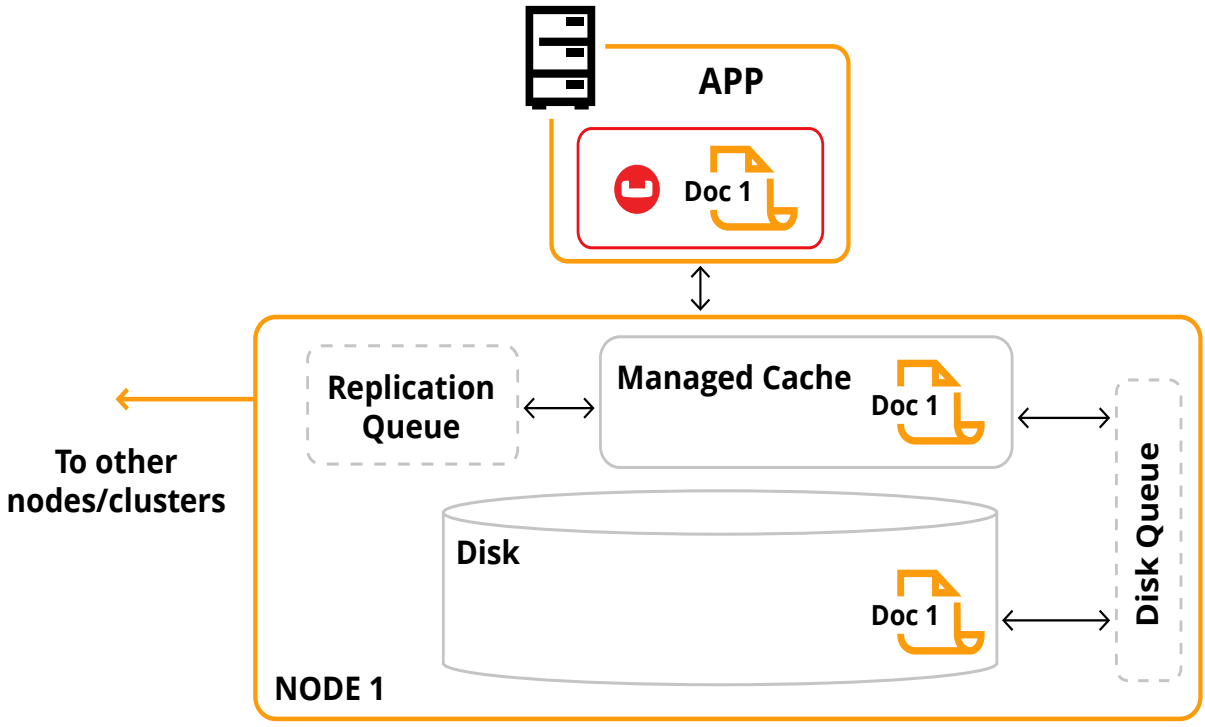
COMPACTION

Couchbase writes all data that you append, update and delete as files on disk. This can eventually lead to gaps in the data file, particularly when you delete data. You can reclaim the empty gaps in all data files by performing a process called compaction. For both data files and index files, perform frequent compaction of the files on disk to help reclaim disk space and reduce disk fragmentation. Auto-compaction is enabled by default for all buckets, but parameters can be adjusted for the entire cluster or for a specific bucket in a cluster.

MUTATIONS

In Couchbase Server, mutations happen at a document level. Clients retrieve the entire document from the server, modify certain fields, and write the document updates back to Couchbase.





When Couchbase receives a request to write a document, the following occurs:

1. Every server hosting the Data Service in a Couchbase cluster has its own managed object cache. The client writes a document into the cache, and the server sends the client a confirmation. By default, the client does not have to wait for the server to persist and replicate the document as it happens asynchronously.
2. The document is added into the intra-cluster replication queue to be replicated to other servers within the cluster.
3. The document is added into the disk-write queue to be asynchronously persisted to disk. The document is persisted to disk after the disk-write queue is flushed.
4. After the document is persisted to disk, it's replicated to other clusters using XDCR and eventually indexed.

Key-value data access

While Couchbase is a document database, at its heart is a distributed key-value (KV) store. A KV store is an extremely simple, schema-less approach to data management that, as the name implies, stores a unique ID (key) together with a piece of arbitrary information (value); it may be thought of as a hash map or dictionary. The KV store itself can accept any data, whether it be a binary blob or a JSON document, and Couchbase features such as the SQL++ query service make use of the KV store's ability to process JSON documents.

Due to their simplicity, KV operations execute with extremely low latency, often sub-millisecond. The KV store is accessed using simple CRUD (Create, Read, Update, Delete) APIs, and provides the simplest interface when accessing documents using their IDs.

The KV store contains the authoritative, most up-to-date state for each item. Query, and other services, provide eventually consistent indexes, but querying the KV store directly will always access the latest version of data. Applications use the KV store when speed, consistency, and simplified access patterns are preferred over flexible query options.

All KV operations are atomic, which means that Read and Update are individual operations. In order to avoid conflicts that might arise with multiple concurrent updates to the same document, applications may make use of Compare-And-Swap (CAS), which is a per-document checksum that Couchbase modifies each time a document is changed.

Data Storage Engines: Magma and Couchstore

Magma is Couchbase's high-density storage engine and the default within version 8. It was designed for both high-speed throughput and very large data sets. The Magma storage engine has both compute and storage separation advantages, performance processing is up to 4x faster while utilizing up to 10x less memory, and storage capacity holds 3x larger data sets per node. (from 3TB to 10TB per cluster node). These advantages result in smaller affordable clusters, holding and processing more data, with higher processing throughput power.

Magma combines the performance of a log-structured merge trees (LSM) with the compaction, reorganizability, and immutability of sorted string tables (SSTables) to provide a high performance in a well organized, low-latency engine that suits write-heavy, low-latency point lookup workloads. This design minimizes disk space increases called "storage amplification" and the accompanying complexity, which occurs when documents are heavily mutated without being reorganized.

The Magma storage engine, within version 8.0, provides customers with the flexibility to utilize either 128 or 1024 vBuckets, with the minimum memory quota of 100 MiB (128 vBuckets) and 1 GiB (1024 vBuckets). This allows customers to better align their storage needs to their use cases.

Query service

The query service is an engine for processing SQL++ queries and follows the same scalability paradigm that all the services use which allows the user to scale query workloads independently of other services as needed.

Couchbase SQL++ (SQL for JSON) combines the flexibility of JSON with the expressive power of SQL. Couchbase SQL++ is an original implementation of the SQL++ standard. The Couchbase team worked with Don Chamberlin, the co-inventor of SQL, as a technical advisor and to build our SQL++ tutorials. SQL++ enables clients to access data from Couchbase using SQL-like language constructs. It includes a familiar data definition language (DDL), data manipulation language (DML), and query language statements, but can operate in the face of NoSQL database features such as key-value storage, multi-valued attributes, and nested objects.



SQL++ provides a rich set of features that let users retrieve, manipulate, transform, and create JSON document data. Its key features include a powerful SELECT statement that extends the functionality of the SQL SELECT statement to work with JSON documents. Of particular importance are the USE KEYS, NEST, and UNNEST sub-clauses of the FROM clause in SQL++ as well as the MISSING Boolean option in the WHERE clause.

The following examples illustrate three sample queries – showing common SQL capabilities and the JSON responses.

SQL++ supports standard SELECT, FROM, WHERE, GROUP BY clauses as well as JOIN capabilities	`SELECT c.name, o.order_date`
	`FROM customers AS c`
	`LEFT OUTER JOIN orders AS o`
	`  ON c.custid = o.custid`
	`WHERE c.custid = "C41";`
		`{`
		` "results": [`
		` {`
		` "name": "R. Duvall",`
		` "order_date": "2017-04-29"`
		` },`
		` {`
		` "name": "R. Duvall",`
		` "order _ date": "2017-09-02"`
		` }`
		` ]`
		`}`

Use UNNEST to extract individual items from a nested JSON array	`SELECT o.orderno,`
	`  i.itemno AS item_number,`
	`  i.qty AS quantity`
	`FROM orders AS o`
	`UNNEST o.items AS i`
	`WHERE i.qty > 100;`
		`{`
		` "results": [`
		` {`
		` "orderno": 1002,`
		` "item _ number": 680,`
		` "quantity": 150`
		` },`
		` {`
		` "orderno": 1005,`
		` "item _ number": 347,`
		` "quantity": 120`
		` },`
		` {`
		` "orderno": 1006,`
		` "item _ number": 460,`
		` "quantity": 120`
		` }`
		` ]`
		`}`

Use MISSING Boolean keyword in WHERE clause to adapt queries when a schema has changed or lacks specific keys	`SELECT o.orderno, SUM(o.cost) AS cost`
	`FROM orders AS o`
	`WHERE o.cost IS NOT MISSING`
	`GROUP BY o.orderno;`
		`{`
		` "results": [`
		` {`
		` "orderno": 1002,`
		` "cost": 220`
		` },`
		` {`
		` "orderno": 1005,`
		` "cost": 623`
		` }`
		` ]`
		`}`



ACID TRANSACTIONS IN SQL++

Couchbase supports the ability to define ACID transactions within SQL++. Transactions can be applied to one or more documents, and can span one or more collections and across cluster nodes. Due to the guarantees required by ACID, transactions may execute more slowly than non-transactional queries in Couchbase, but this flexibility is available to the developer. Transactions in SQL++ have adopted a nearly identical syntax to SQL for relational databases. Transactions can also be built into and managed by the application using the Java SDK.

COMPARE SQL TRANSACTIONS TO COUCHBASE SQL++

Typical RDBMS

```
START TRANSACTION;
UPDATE customer SET=balance + 100 WHERE cid = 4872;
SELECT cid, name, balance FROM customer;
SAVEPOINT s1;
UPDATE customer SET balance = balance – 100 WHERE cid = 1924;
SELECT cid, name, balance FROM customer;
ROLLBACK WORK TO SAVEPOINT s1;
SELECT cid, name, balance FROM customer;
COMMIT;
```

Couchbase

```
START TRANSACTION;
UPDATE customer SET balance = balance + 100 WHERE cid = 4872;
SELECT cid, name, balance FROM customer;
SAVEPOINT s1;
UPDATE customer SET balance = balance – 100 WHERE cid = 1924;
SELECT cid, name, balance FROM customer;
ROLLBACK WORK TO SAVEPOINT s1;
SELECT cid, name, balance FROM customer;
COMMIT;
```

COST-BASED QUERY OPTIMIZATION

The query service uses a cost-based query optimizer to take advantage of indexes that are available. Index nodes can handle much of the data aggregation pipeline as well, so that less data is sent back to the query node for processing. Cost-based optimization for queries accessing JSON structures was patented by Couchbase in 2021.

QUERY AUTOMATIC WORKLOAD REPOSITORY

Within version 8.0, Couchbase Enterprise can collect all query execution statistics for summary reports for queries' elapsed time, CPU and memory usage, KV fetch, executions, and other metrics. The collected stats are written to a user-provided collection, allowing users to determine the retention period. A new reporting tool allows users to generate summary reports from the stats for any period of time for which the query workload had been enabled for. The statistics are aggregated and captured per unique query, identifiable by a SQLID – this allows users to perform further analysis using the systems:completed_requests

Capella iQ and developer tools

Capella iQ – A generative AI-powered coding assistant that's built into the Capella Workbench and popular IDEs to help developers instantly become more productive. Developers can converse with the Capella iQ coding assistant to create datasets, compose SQL++ queries, build indexes, define text searches, write application-specific code, and build and run functional test cases. Additionally, new with 8.0 is natural language support for the Query service. Customers can either use Couchbase Command Line Shell for SQL++ or the Query Workbench. Note, a Capella account (either free or paid) is needed to utilize this feature.

- Couchbase Shell – A modern shell to interact with Couchbase Server and Cloud
- Capella DataStudio – A community-supported, free, cool data management tool with a single pane of glass UI for Capella Operational, Capella Columnar and Couchbase Server Clusters
- Couchbase SDKs – Allow applications to access a Couchbase cluster and the big data Connectors enable data exchange with other platforms.
- Couchbase SDK Doctor – A command-line tool for Couchbase Server SDKs that diagnoses network connectivity issues, verifies configurations, and tests for common errors. It runs on Windows, macOS, and Linux to help developers and admins troubleshoot and ensure proper SDK usage.
- Ottoman – an Object Data Modeler (ODM) for Couchbase's Node.js SDK, providing JSON schema and validation for NoSQL.



Index service

Secondary indexing is an important part of making queries run efficiently and Couchbase provides a robust set of index types and management options. The index service is responsible for all of the maintenance and management tasks of indexes, known as Global Secondary Indexes (GSI). The index service monitors document mutations to keep indexes up to date, using the database change protocol stream (DCP) from the data service. It is distinct from the query service, allowing their workloads to be isolated from one another where needed.

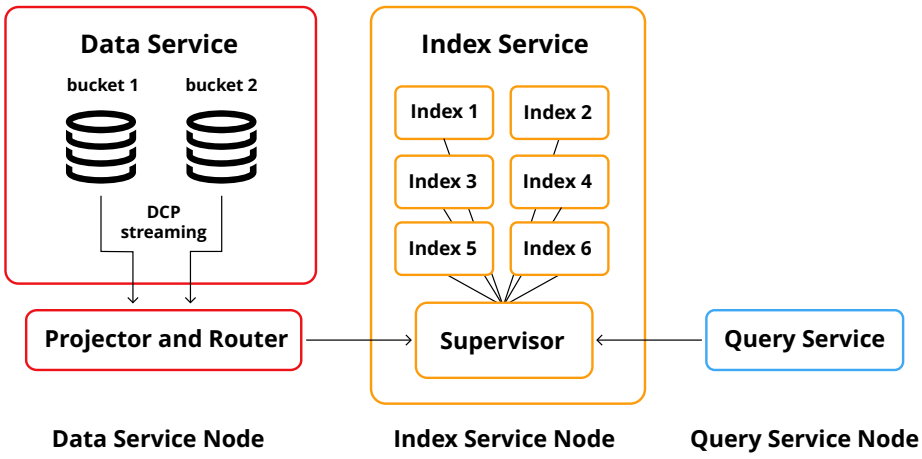
The following is a sample of some of the types of indexes supported by the index service:

- **Primary –** indexes whole bucket or collection using the document key
- **Secondary –** indexes a scalar, object, or array using a key-value
- **Composite/Covered –** multiple fields stored in an index or an array index
- **Functional –** secondary index that allows functional expressions instead of a simple key-value
- **Array –** an index of array elements ranging from plain scalar values to complex arrays or JSON objects nested deeper in the array

- **Adaptive –** secondary array index for all or some fields of a document without having to define them ahead of time
- **Flex –** Flex indexes are used for queries containing compound selection criteria such as those derived from form-base applications with multi-select options. Flex Index uses the inverted indexes from the Full-Text Search engine to accommodate unknown values.

INDEX ADVISOR

Is a built-in query command, ADVICE, that interrogates the database for which index or GSI to use or build given the object and predicate selections contained in the query statement.



Query consistency

Under the hood, Couchbase indexes are updated asynchronously after the data has been changed by the application. In comparison to other database technologies, this allows for much higher write throughput but introduces the possibility of inconsistency between the data and its indexes. Couchbase, therefore, provides several levels of control over query consistency, allowing the application to choose between faster queries (ignoring pending mutations) and stronger consistency.

The following consistency levels are specified per-query, allowing for even finer grained and dynamic control of these tradeoffs:

- **not_bounded (default) –** Return the query response immediately, knowing that some data may still be flowing through the system. This consistency level is useful for queries that favor low latency over consistency.
- **at_plus –** Block the query until its indexes have been updated to the timestamp of the last update, knowing that some updates may have happened since but don't need to be consistent. This is for "read-your-own-write" semantics by a single application process/thread.

- **request_plus –** Block the query until its indexes are updated to the timestamp of the current query request. This is a strongly consistent query and is used to maintain consistency across applications/processes.

Indexes are updated as fast as possible, regardless of query consistency requirements. Even a query requesting strong consistency may return extremely quickly if its indexes are not processing large volumes of updates.

MEMORY-OPTIMIZED INDEXES (MOI)

Memory-optimized indexes use a skiplist structure as opposed to B-tree indexes, optimizing memory consumption and concurrent processing of index updates and scans. MOI provides the most optimized index for processing high-velocity mutations and high-frequency scans. MOI is essentially "in-memory" and therefore requires enough RAM to store all indexes.

Full-Text Search

The Search service is an engine for performing Full-Text Searches (FTS) on the JSON data stored within a bucket or a collection. FTS lets you create, manage, and query inverted indexes for searching of free-form text within a document. The service provides analyzers that perform several types of operations including multi-language tokenization, stemming, and relevance scoring. Search supports synonyms, index-level filtering, BM25 scoring, and reading replicas for better performance.

Search nodes incorporate both an indexer and query processor, much like the query and index services, except these don't run on separate nodes – both workloads run on each search node.

As with the other services, data nodes use the DCP stream to send mutations to the FTS indexer process for index updating whenever data changes. Index creation is highly configurable through a JSON index definition file, Couchbase SDK, or through a graphical web interface as part of the administration console.

Documents can be indexed differently depending on a document type attribute, a document ID, or the value of a designated attribute. Each index definition can be assigned its own set of analyzers and specific analyzers can be applied to indexes for a subset of fields.

Indexes are tied to a specific bucket or collection, but it is possible to create virtual index aliases that combine indexes from multiple buckets or collections into a single seamless index. These aliases also allow application developers to build new indexes and quickly change over to new ones without having to take an index offline.

Searching and indexing use the same set of analyzers for finding matching data. All data, when indexed, flows through the analyzer steps as defined by the index. Then search requests are received and passed through the same steps – for example, tokenization, removing stop words, and stemming terms. These analyzed search requests are then looked up by the indexer in the index and matches are returned. The results include the source request, list of document IDs, and relevance scoring.

Other indexing and search-time options provide fine-grained control over indexing more or less information depending on the use case. For example, text snippets may also be stored in the index and included in the search response so that retrieving full documents from the data service is not required.

Couchbase developed Bleve, the open source, Go-based search project, for the FTS capabilities, including language support, scoring, etc.

Geospatial search

Geospatial search in Couchbase enables efficient querying of location-based data, allowing applications to retrieve and filter information based on spatial coordinates, distances, and geographic relationships. This functionality is particularly powerful for use cases such as local search, real-time location services, and logistics optimization, where pinpointing and ranking data by proximity is critical. By integrating geospatial (or geographic) search capabilities directly into the database, developers can enhance experiences like finding nearby stores, optimizing delivery routes, or enabling map-based searches with high accuracy and performance.

Couchbase supports various types of geometries and geospatial queries to enable flexible and powerful location-based searches. Here are the key types available:

GEOMETRIES:

- **Point:** Represents a single location defined by latitude and longitude coordinates.
- **LineString:** A series of connected points that form a path or a route.
- **Polygon:** A closed shape defined by multiple points, used to represent areas like boundaries or regions.
- **MultiPoint:** A collection of points.
- **MultiLineString:** A collection of LineStrings.
- **MultiPolygon:** A collection of Polygons.

GEOSPATIAL QUERIES:

- **Bounding Box Query:** Filters results that fall within a specified rectangular area defined by minimum and maximum latitude/longitude values.
- **Distance Query:** Retrieves documents where geometries are within a specified radius from a given point.
- **Polygon Query:** Filters results based on whether they fall within a specified polygon shape.
- **Point Query:** Retrieves documents that match a specific geographic point.
- **Geospatial Range Query:** Combines distance and bounding box queries for more complex location filtering.



The steps to using geospatial search queries are straightforward and can be easily combined with other types of search queries:

- Add GeoJSON data to your documents to represent geographical points, lines, and polygons.
- Create indexes that support geospatial data for efficient searching by location and proximity.
- Run queries using spatial filters, such as bounding boxes or distance-based queries, to retrieve relevant location-based results.

Vector search

Vector search gives users more powerful natural language ways to interact with data and applications. Common use cases for organizations are powerful, natural language chatbots, sophisticated enterprise search that delivers a hybrid search combining range, text, and vector predicates, and data analysis spotting similarity and anomalies.

Vector search delivers nearest-neighbor results, without needing a direct match. Text, images, audio, and video are converted to mathematical representations and used for semantic searching or overcoming GenAI challenges using the retrieval-augmented generation (RAG) framework.

You can develop applications that include:

Similarity search – Similarity is a powerful tool for users to find products and information, but many real-world scenarios have users wanting to search across a variety of methods, like text, geolocations, ranges, and include operational data too. By using vector embeddings, you can search based on descriptions, rather than using specific keywords and get intuitive and relevant results across data types. Couchbase lets developers build powerful search functionality to delight users.

Hybrid search – Hybrid search is usually the combination of Vector (semantic/similarity) + Keyword (lexical) searches at the same time. We extend that to include more search types: ranges, predicates (where clauses), calculated values (columnar), geo-location (FTS), etc.

Semantic search – Use natural language processing to deliver more accurate results based on an understanding of the intent and context behind a given search query, rather than simple keyword matches.

Generative AI – Create new original content, such as text and images, based on a prompt or a vector search given to an LLM. Use generative AI to get tailored and dynamic responses across your applications.

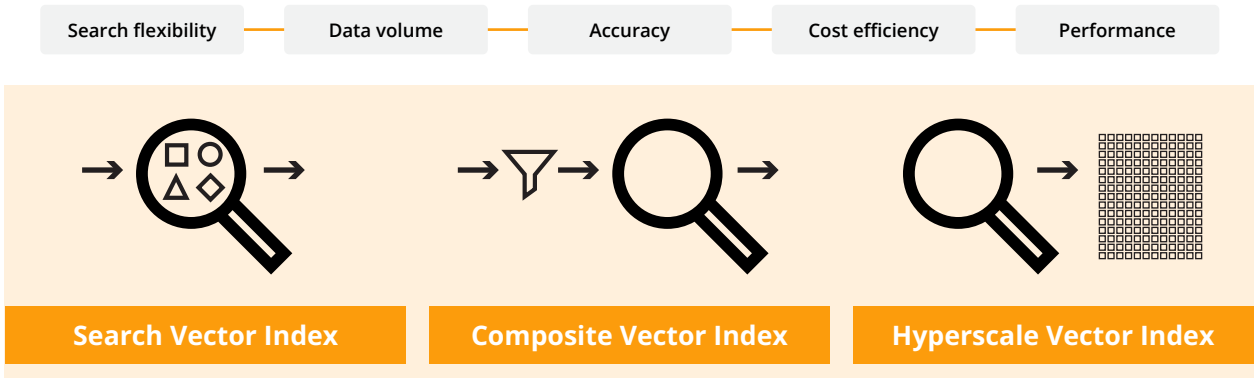
Couchbase provides three types of vector search indexes that benefit customers to align the best index to match their performance and efficiency needs. These include:

- **Search Vector Index:** This is our original vector index in the Search service and its power lies in its ability to perform hybrid search. Teams can combine text search, geospatial filters, and vector similarity all in a single query. This is the go-to for complex, multi-faceted searches, like finding “Italian restaurants near me with a similar vibe to this photo.” It can handle all these different data types and query types at once.



- **Composite Vector Index:** This index is part of our Index service and is designed to make queries incredibly fast when data needs pre-filtering. For example, "find the best-selling products in the last month that are similar to this item." It's about precision and efficiency.
- **Hyperscale Vector Index:** This index is about raw power, speed, and performance at scale. It is best utilized with a very large number of vectors (even billions) and you need top-tier throughput, the lowest possible latency, and the highest recall. An example would be large scale content recommendation engines, "Find me 100 products similar to this one" or fraud detection analyzing a massive volume of data in real time. The hyperscale index uses a patented storage algorithm.

Choose based on workload priorities



Eventing service

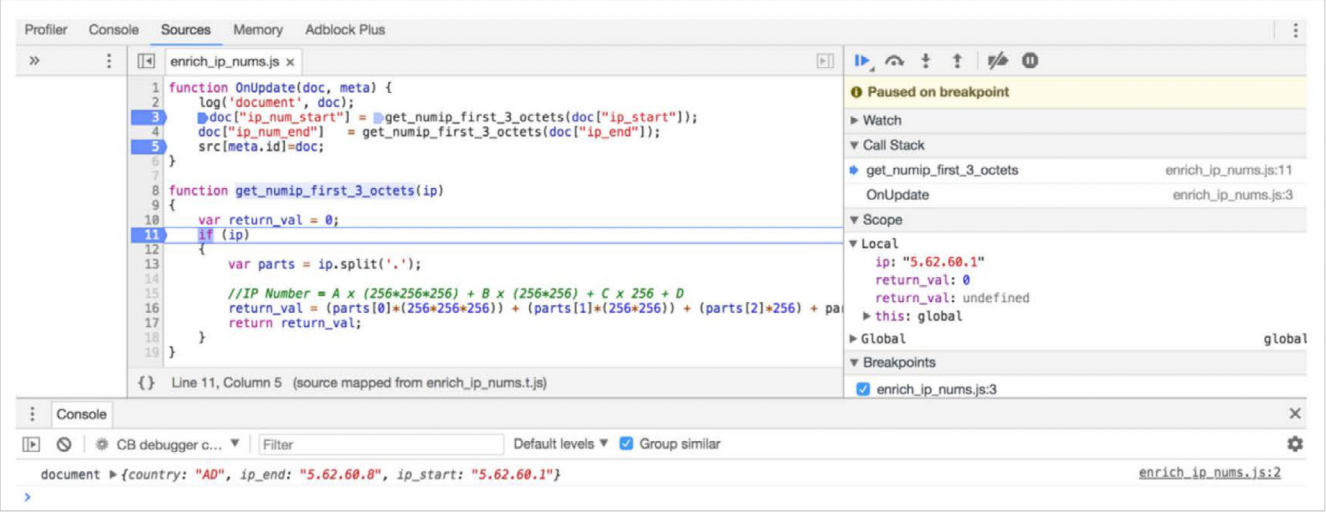
The eventing service supports custom server-side functions (written in JavaScript) that are automatically triggered using an Event-Condition-Action model. These functions receive data from the DCP stream for a particular bucket or collection and execute code when triggered by data mutations. Similar to other services, the eventing service scales linearly and independently.

Eventing service offers both "change data capture" like features found in event handlers as well as multi-channel data streaming features found in solutions such as Apache Kafka. Code processes the source data and commits it as a new or updated document in another bucket.

The core of eventing functions is a Google V8 execution container. Functions inherit support for most of the standard ECMAScript constructs that are available through V8. Some capabilities have been removed to support the ability to shard and scale execution automatically. Additionally, to optimize language utilization of the server environment, some new constructs have been added.

Code for functions is written in a web-based JavaScript code editor and features an extensive in-browser debugging environment.

With 8.0, Couchbase introduces the OnDeploy Handler. This enables custom actions to run automatically whenever an Eventing function is deployed or redeployed. The OnDeploy Handler allows for automated setup, meaning critical initialization and data validation can happen right at deployment. Plus, it offers increased flexibility because the source bucket is now optional, broadening how Eventing functions can be used for general automation. This gives users enhanced control with a dedicated hook for pre-execution logic, reducing manual work and potential errors.



DISTRIBUTED FOUNDATION

The foundation of Couchbase is a clustering approach that provides flexible options for scaling while maintaining performance and availability. This scaling approach is applied across all levels of the cluster, providing high-performance flexibility for nodes, services, buckets, and vBuckets, scopes, and collections.

Node-level architecture

A Couchbase cluster consists of a group of interchangeable, largely self-sufficient nodes that operate in a peer-to-peer topology. There is just one Couchbase node type, though the services running on that node can be managed as required (see Multi-Dimensional Scaling).

Having a single-node type greatly simplifies the installation, configuration, management, and troubleshooting of a Couchbase cluster, both in terms of what you must do as a human operator and what the automatic management needs to do. There is no concept of master nodes, slave nodes, config nodes, name nodes, or head nodes.

Components of a Couchbase node include the cluster manager and, optionally, the data, query, index, analytics, search, eventing and backup services. There are also the underlying managed cache and storage components. By dividing up potentially conflicting workloads in this way, a Couchbase node can achieve maximum throughput and resource utilization and minimum latency.

Nodes can be added or removed easily through a rebalance process, which redistributes the data evenly across all nodes. The rebalance process is done online and requires no application downtime, and can be initiated at the click of a button or one command on the command line.

Couchbase Server is typically deployed on a cluster of commodity servers, virtual machines or cloud instances, although for development purposes, all functionality can be run on a single node. An application can be developed on a small scale, even on a laptop, and then deployed to a distributed cluster without any architecture or behavioral changes to that application.

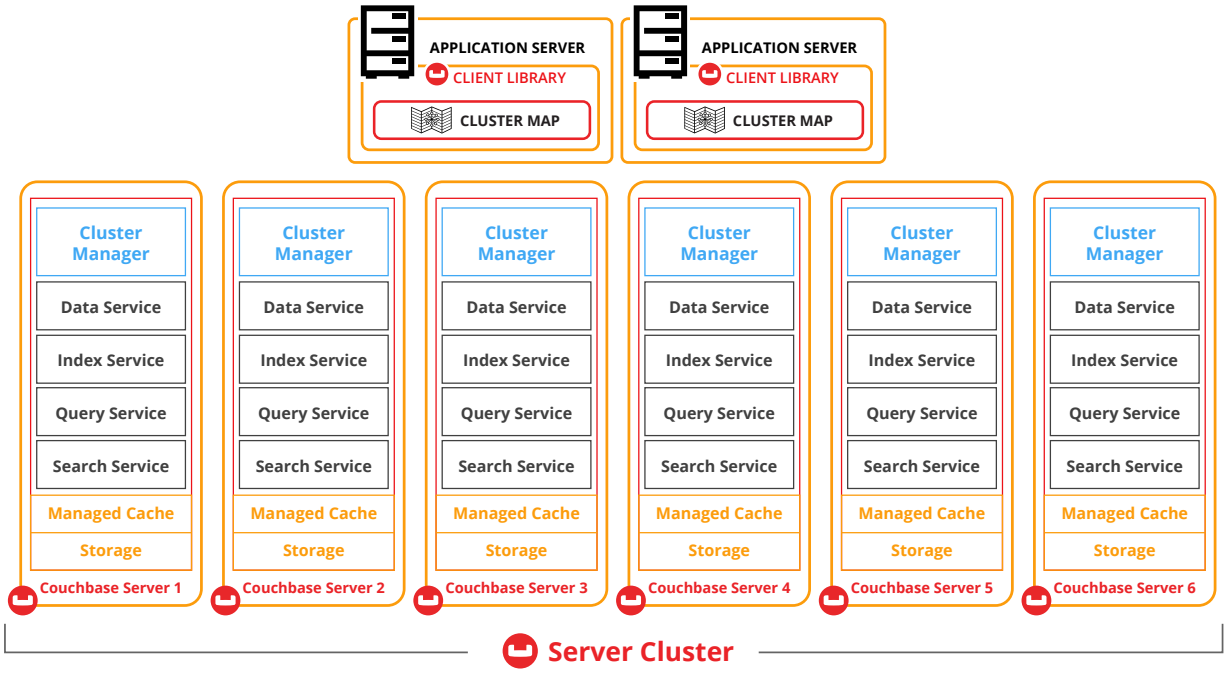
Capacity can be increased or decreased simply by adding or removing nodes. In this way, a cluster can grow CPU, RAM, disk, and network capacity by adding physical servers or virtual machines that have the exact same software installed.

The maximum capacity managed by a single node can be up to 10TB using a high-density storage engine and allowing the storage capacity to exceed the node's memory size. Memory sizing and allocation also affect the processing capacity of a node, as do the performance requirements of the services it hosts. The maximum manageable capacity of nodes running versions earlier than Couchbase Server 7.1 may support up to 3TB of data.

Cluster architecture

A cluster consists of one or more instances of Couchbase Server, each running on an independent node. Data and services are shared across the cluster.

The following figure shows application servers communicating with the cluster overall, but because they are also aware of the individual node topology, they can adapt as needed. For example, if a replica needs to be read, the application server can access it directly because it knows about the overall cluster configuration.



The various services that Couchbase provides are also fed data through the DCP stream, which is internally used for sending new/changed data to these services as well as providing the basis for keeping data in sync between nodes in the cluster.

CLUSTER/NODE CONFIGURATION

When Couchbase is being configured on a node, it can be specified either as its own, new cluster, or as a participant in an existing cluster. Thus, once a cluster exists, successive nodes can be added to it. When a cluster has multiple nodes, the Couchbase cluster manager runs on each node: this manages communication between nodes, and ensures that all nodes are healthy.

Services can be configured to run on all or some nodes in a cluster, and can be added/removed as warranted by established performance needs. For example, given a cluster of five nodes, a small dataset might require the data service on only one of the nodes; a large dataset might require four or five. Alternatively, a heavy query workload might require the query service to run on multiple nodes, rather than just one. This ability to scale services individually promotes optimal hardware resource utilization.

CLUSTER MANAGER

The cluster manager supervises server configuration and interaction between servers within a Couchbase cluster. It is a critical component that manages replication and rebalancing operations in Couchbase. Although the cluster manager executes locally on each cluster node, it elects a clusterwide orchestrator node to oversee cluster conditions and carry out appropriate cluster management functions.

If a machine in the cluster crashes or becomes unavailable, the cluster orchestrator notifies all other machines in the cluster, and promotes to active status all the replica partitions associated with the server that's down. The cluster map is updated on all the cluster nodes and the clients. This process of activating the replicas is known as failover. You can configure failover to be automatic or manual. Additionally, you can trigger failover through external monitoring scripts via the REST API.

If the orchestrator node crashes, existing nodes will detect that it is no longer available and will elect a new orchestrator immediately so that the cluster continues to operate without disruption.

In addition to the cluster orchestrator, there are three primary cluster manager components on each Couchbase node:

The heartbeat watchdog – periodically communicates with the cluster orchestrator using the heartbeat protocol, providing regular health updates for the server. If the orchestrator crashes, existing cluster server nodes will detect the failed orchestrator and elect a new orchestrator.

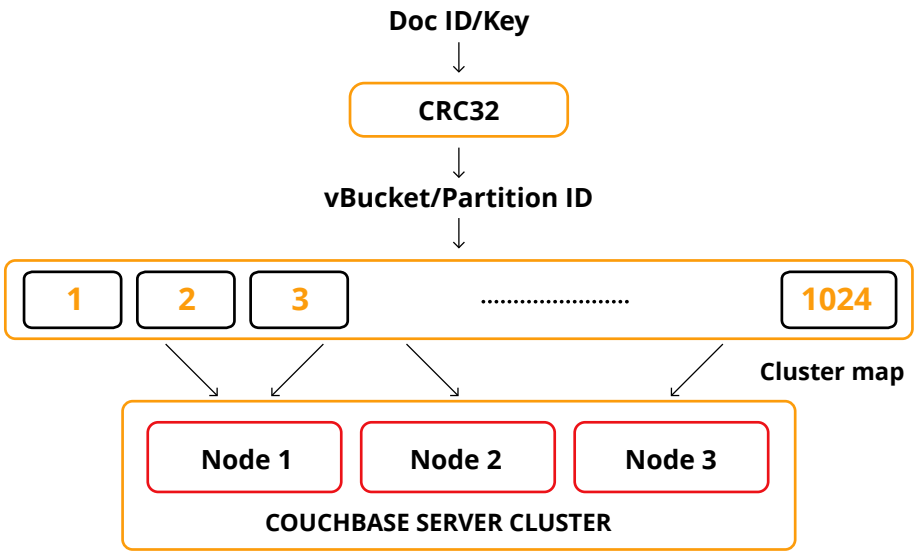
The process monitor – monitors local data manager activities, restarts failed processes as required, and contributes status information to the heartbeat process.

The configuration manager – receives, processes, and monitors a node's local configuration. It controls the cluster map and active replication streams. When the cluster starts, the configuration manager pulls the configuration of other cluster nodes and updates its local copy.

Client connectivity

To talk to all the services of a cluster, applications use the Couchbase SDK. Support is available for a variety of languages, including Java, Scala, .NET, PHP, Python, Go, Node.js, and C/C++. These clients are continually aware of the cluster topology through cluster map updates from the cluster manager. They automatically send requests from applications to the appropriate nodes for KV access, query, etc.

When creating documents, clients apply a hash function (CRC32) to every document that needs to be stored in Couchbase, and the document is sent to the server where it should reside. Because a common hash function is used, it is always possible for a client to determine on which node the source document can be found.



TOPOLOGY-AWARE CLIENT

After a client first connects to the cluster, it requests the cluster map from the Couchbase cluster and maintains an open connection with the server for streaming updates. The cluster map is shared with all the servers in a Couchbase cluster and with the Couchbase clients. Data flows from a client to the server using the following steps:

1. An application interacts with an application, resulting in the need to update or retrieve a document in Couchbase Server.
2. The application server contacts Couchbase Server via the smart client SDKs.
3. The client SDK takes the document that needs to be updated and hashes its document ID to a partition ID. With the partition ID and the cluster map, the client can figure out on which server and on which partition this document belongs. The client can then update the document on this server.
4. When a document arrives in a cluster, Couchbase Server replicates the document, caches it in memory and asynchronously stores it on disk.

APPLICATION SERVICE-LEVEL TELEMETRY

Couchbase 8.0 lets the cluster collect metrics from SDK clients – not just from the server. This gives visibility into timeouts, cancellations, and latency from the application's point of view. This is beneficial for spotting issues like slow networks or client hiccups. Teams can access these metrics through the native Prometheus endpoint, the Stats API, or even in standard log bundles – making it easy to monitor or share with Support when needed. Metrics are aggregated across clients to mitigate metrics explosion. Clients can send metrics to any node and the server will forward appropriately.

Data transport via Database Change Protocol (DCP)

DCP is the protocol used to stream bucket-level mutations. It is used for high-speed replication of data as it mutates – to maintain replica vBuckets, global secondary indexes, full-text search, eventing, ETL and analytic writeback, XDCR, and backups. Connectors to external services, such as Elasticsearch, Spark, or Kafka are also fed from the DCP stream.

DCP is a memory-based replication protocol that is *ordering, resumable, and consistent*. DCP stream changes are made in memory to items by means of a replication queue.

An external application client sends the operation requests (read, write, update, delete, query) to access or update data on the cluster. These clients can then receive or send data to DCP processes running on the cluster. External data connectors, for example, often sit and wait for DCP to start sending the stream of their data when mutations start to occur.

Whereas an internal DCP client, used by the cluster itself, streams data between nodes to support replication, indexing, ingestion, writeback, cross datacenter replication, incremental backup, and mobile synchronization. Sequence numbers are used to track each mutation in a given vBucket, providing a means to access data in an ordered manner or to resume from a given point in time.

Multi-Dimensional Scaling (MDS)

Couchbase MDS features improve performance and throughput for mission-critical systems by enabling independent scaling of data, query, and indexing workloads. Scale-out and scale-up are the two scalability models typical for databases – Couchbase takes advantage of both. There are unique ways to combine and mix these models in a single cluster to maximize throughput and latencies. With MDS, admins can achieve both the existing homogeneous scalability model and the newer independent scalability model.

HOMOGENEOUS SCALING MODEL

To better understand the multi-dimensional scaling model, it is beneficial to take a look at a typical homogeneous scaling model.

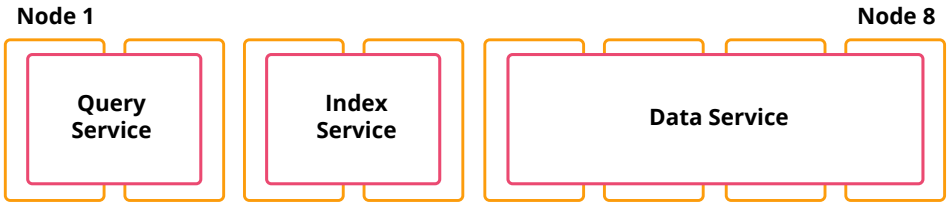


In this model, application workloads are distributed equally across a cluster made up of a homogeneous set of nodes. Each node that does the core processing takes a similar slice of the work and has the same hardware resources.

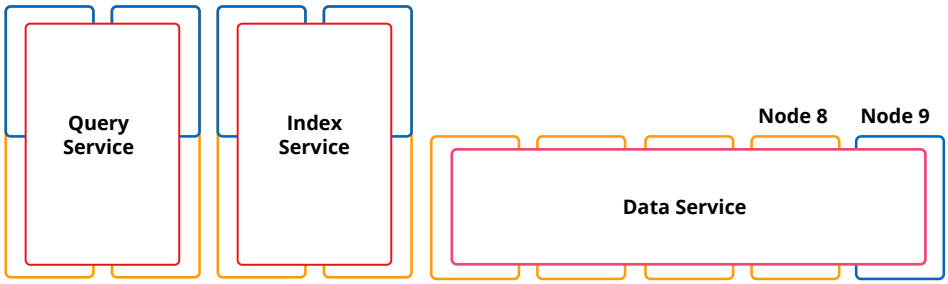
This model is available through MDS and is simple to implement but has a couple of drawbacks. Components processing core data operations, index maintenance, or executing queries all compete with each other for resources. It is impossible to fine-tune each component because each of them has different demands on hardware resources. This is a common problem with other NoSQL databases. While the core data operations can benefit greatly from scale-out with smaller commodity nodes, many low latency queries do not always benefit from wider fan-out.

Independent scaling model

MDS is designed to minimize interference between services. When you separate the competing workloads into independent services and isolate them from each other, interference among them is minimized. The figure below demonstrates a deployment topology that can be achieved with MDS. In this topology, each service is deployed to an independent zone within the cluster.



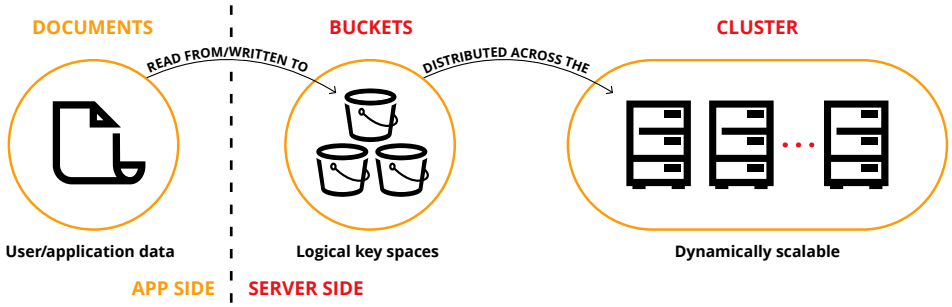
Each service zone within a cluster (data, query, and index services) can now scale independently so that the best computational capacity is provided for each of them.



In the figure above, the blue additions signify the direction of scaling for each service. In this case, query and index services scale-up over the fewer sets of powerful nodes and data service scales out with an additional node.

Data distribution

Couchbase partitions data into vBuckets (synonymous to shards or partitions) to automatically distribute data across nodes, a process sometimes known as auto-sharding.



vBuckets help enable data replication, failover, and dynamic cluster reconfiguration. Unlike data buckets, users and applications do not manipulate vBuckets directly. Couchbase automatically divides each bucket into 1024 active vBuckets and 1024 replica vBuckets per replica, and then distributes them evenly across the nodes running the data service within a cluster. vBuckets do not have a fixed physical location on nodes; therefore, there is a mapping of vBuckets to nodes known as the cluster map. Through the Couchbase SDK, the application automatically and transparently distributes the data and workload across these vBuckets.

INDEX PARTITIONS AND REPLICAS

Global secondary indexes (GSI) can also be partitioned using several approaches depending on the pattern of queries that need to be supported. Commonly, GSI are stored identically on each index node so that any node can help with queries. But in some cases it is better for performance or storage management to spread the indexes across several nodes. There are many options for creating these partitions – different attributes in a document can be used as a hash key (one or more), partitions can also be assigned to specific nodes, and more.

As documents are added and updated, those mutations are streamed from memory to a local projector process, which then forwards them to the relevant index service node(s). Every index node has a supervisor process running the index service. This process listens to changes from the projector processes on the data nodes. It evaluates the incoming stream of changes for the specific indexes created on the node running the index service. Each index is then updated independently with that data.

Individual indexes can be automatically replicated to other nodes in the cluster to achieve high availability, ensuring that an index continues to function even if a node hosting the index is unavailable. Queries will load balance across the indexes and if one of the indexes become unavailable, all requests are automatically rerouted to the available remaining index without application or admin intervention.

Similarly, Couchbase supports independent partitioning of indexes to distribute the data volumes and load of a single index across multiple processes and/or nodes.

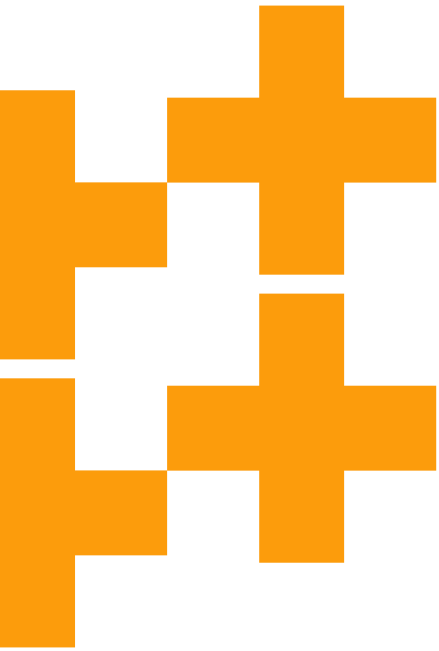
PARTITIONING OTHER SERVICES

All Couchbase services partition data and workloads across available nodes. Full-text search, eventing, and replicating data and processes. They also all redistribute these across new cluster topologies when rebalancing occurs.

- **Search service –** automatically partitions its indexes across all search nodes in the cluster, ensuring that during rebalance, the distribution across all nodes is balanced
- **Eventing service –** vBucket processing ownership is distributed across eventing nodes

Rebalancing the cluster

When the number of servers in the cluster changes due to scaling out or node failures, data partitions must be redistributed. This ensures that data is evenly distributed across the cluster, and that application access to the data is load-balanced evenly across all the servers. This process is called rebalancing. All Couchbase services are rebalance-aware and follow their own set of internal processes for rebalance as needed.



Rebalancing is triggered using an explicit action from the admin web UI or through a REST call. When initiated, the rebalance orchestrator calculates a new cluster map based on the current pending set of servers to be added and removed from the cluster. It streams the cluster map to all the servers in the cluster. During rebalance, the cluster moves data via partition migration directly between two server nodes in the cluster. As the cluster moves each partition from one location to another, an atomic and consistent switchover takes place between the two nodes, and the cluster updates each connected client library with a current cluster map.

Throughout migration and redistribution of partitions among servers, any given partition on a server will be in one of three states:

- **Active –** the server hosting the partition is servicing all requests for this partition.
- **Replica –** the server hosting the partition cannot handle client requests, but can receive replication commands. Rebalance marks destination partitions as replica until they are ready to be switched to active.
- **Dead –** the server is not in any way responsible for this partition.

The node health monitor receives heartbeat updates from individual nodes in the cluster, updating configuration and raising alerts as required. The partition state and replication manager is responsible for establishing and monitoring the current network of replication streams.



High availability

"Couch," in name Couchbase, is actually an original acronym that stands for, "Clusters Of Unreliable Commodity Hardware." We highlight this to remind readers that scalability, reliability and performance are foundational characteristics of the database. To meet high availability requirements, all Couchbase maintenance operations can be done while the system remains online, without requiring modifications or interrupting running applications. The system never needs to be taken offline for routine maintenance such as software upgrades, data rebalancing, index building, compaction, instance refreshes, or any other operation. Even provisioning or removing nodes can be done entirely online without any interruption to running applications, and without requiring developers to modify their applications.

Built-in fault tolerance mechanisms protect against downtime caused by arbitrary unplanned incidents, including server failures. Replication and failover are important mechanisms that increase system availability. Couchbase replicates data across multiple nodes to support failover and durability. Ensuring that additional copies of the data are available is automated to deal with the inevitable failures that large distributed systems are designed to recover from. All of this is done automatically without need for manual intervention or downtime.

COUCHBASE AND CAP THEOREM

The CAP theorem states that a database cannot simultaneously provide all three of the following guarantees:

- Consistency (the latest information is always available everywhere)
- Availability (every read and write request receives a response)
- Partitioning Tolerance (data is persisted upon multiple independent drives for fault tolerance)

Practically speaking, most NoSQL databases are forced to choose whether to favor consistency or availability in specific scenarios, and many offer tunable solutions. Couchbase is usually considered as a CP system, especially when deployed as a single cluster--strongly consistent with durable, redundant data partitions. And transactional consistency can be guaranteed when choosing the option to use ACID transactions within the cluster. Couchbase holds patents for multi-document distributed ACID transactions for JSON documents.

However, when creating geographically dispersed, edge, or mobile systems, Couchbase can be configured to favor AP when ensuring local data availability and durability are more important than systemic consistency. The timing window of this trade-off of allowing a consistency gap will most often be resolved in milliseconds, depending on distance and global network speeds. Given the propensity for networks to fail, however, even vendors that assert to support ACID transactions across global networks cannot guarantee constant global consistency. Many Fortune 100 companies run distributed financial transactions over XDCR and Couchbase synchronization services.

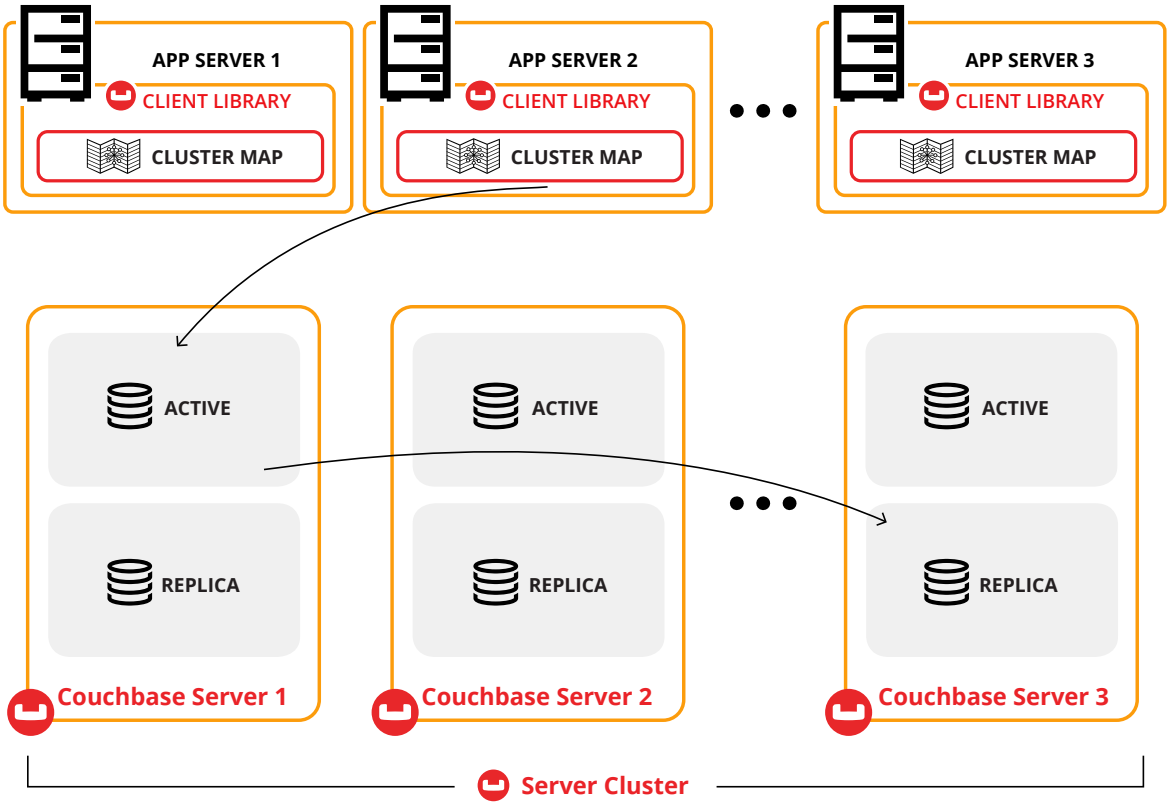


INTRA-CLUSTER REPLICATION

Up to three replica buckets can be defined for every bucket. Each replica itself is also implemented as 1024 vBuckets. A vBucket that is part of the original implementation of the defined bucket is referred to as an active vBucket. Therefore, a bucket defined with two replicas has 1024 active vBuckets and replica vBuckets. Typically, only active vBuckets are accessed for read and write operations: although vBuckets are able to support read requests. Nevertheless, vBuckets receive a continuous stream of mutations from the active vBucket by means of DCP, and are thereby kept constantly up to date.

To ensure maximum availability of data in case of node failures, the master services for the cluster calculate and implement the optimal vBucket distribution across available nodes: consequently, the chance of data loss through the failure of an individual node is minimized, since replicas are available on the nodes that remain.

Active and replica vBuckets correspond to a single, user-defined bucket, for which a single replication instance has been specified. No replica resides on the same node as its active equivalent.



When a node becomes unavailable, failover can be performed and the cluster manager is instructed to read and write data only on available nodes. Failover can be performed by manual intervention or automatically, promoting replica vBuckets to active status when needed. Automatic failover can be performed for one node at a time, and only up to a configurable number of times, the maximum being three. And multi-node failover can be configured to tolerate larger scale failures such as multiple nodes in a racks or VMs on a host all fail at once, failover can switch to vBuckets or collections on an available node.

The cluster manager never performs automatic failover, where data loss might result. The number of times a failover can be safely performed depends on how many nodes and replicas exist. For example, in a five-node cluster with one replica, a single node can be failed over without danger, but if a second node fails, failover might result in data loss, due to required replicas no longer being available. Similarly, in a five-node cluster with two replicas, two nodes can be failed over without danger, but a third cannot. In such cases, a manual recovery process is required. Failures of multiple nodes or instances can be configured based on the number of failures that can be tolerated.

NODE FAILOVER

Failover is the process by which a node of a Couchbase cluster is removed quickly, as opposed to intentional removal and rebalancing.

Auto-failover allows unresponsive servers to be failed over automatically by the cluster manager. Data partitions in Couchbase are always served from a single master node. As a result, if that node is down, the data will not be available until restored. The server will either need to be manually or automatically failed over in order to promote replica data partitions on replica servers to active data partitions that become a new master, so that they can be accessed by the application.

The administrator will not always be able to manually fail servers over quickly enough to avoid application downtime, so Couchbase provides an auto-failover feature. This feature allows the cluster manager to automatically fail over nodes that are down and bring the cluster back to a healthy state as quickly as possible.

In Couchbase Enterprise, nodes can also be automatically failed over when the data service reports sustained disk I/O failures.

FAILOVER CHOICES

As a node failover has the potential to reduce the performance of your cluster, you should consider how best to handle a failed node situation and also size your cluster to plan for failover.

Manual or monitored failover

Manual failover is performed by either human monitoring or by using a system external to the cluster. An external monitoring system can monitor both the cluster and the node environment so that you can make a more data-driven decision.

Human intervention
Humans are uniquely capable of considering a wide range of data, observations, and experiences to resolve a situation in the best possible way. Many organizations disallow automated failover because they want a human to consider the implications. Human intervention tends to be slower than using a computer-based monitoring system.

External monitoring
Another option is to have a system monitoring the cluster via the Couchbase REST API. Such an external system can failover nodes successfully because it can take into account system components that are outside the scope of Couchbase Server.

For example, monitoring software can observe that a network switch is failing and that there is a dependency on that switch by the Couchbase cluster. The system can determine that failing nodes will not help the situation and will, therefore, not failover the node. The monitoring system can also determine if the components around Couchbase Server are functioning and if the various nodes in the cluster are healthy.

If the monitoring system determines the problem is only with a single node and remaining nodes in the cluster can support aggregate traffic, then the system may safely failover the node using the REST API or command-line tools.

Automatic failover
The cluster manager handles the detection, determination, and initiation of the processes to failover a node without user intervention. Once the problem has been identified and fixed, it still requires you to initiate a rebalance to return the cluster to a healthy state.



SERVER GROUP AWARENESS

Server group awareness provides enhanced availability. Specifically, it protects a cluster from large-scale infrastructure failure through the definition of groups. Each group is created by an appropriately authorized administrator and specified to contain a subset of the nodes within a Couchbase cluster. Following group definition and rebalance, the active vBuckets for any defined bucket are located on one group, while the corresponding replicas are located on another group. This allows group failover to be enabled, so that if an entire group goes offline, its replica vBuckets, which remain available on another group, can be automatically promoted to active status.

Groups should be defined in accordance with the physical distribution of nodes. For example, a group should only include the nodes that are in a single server rack, or in the case of cloud deployments, a single availability zone. Thus, if the server rack or availability zone becomes unavailable due to a power or network failure, group failover can allow continued access to the affected data.

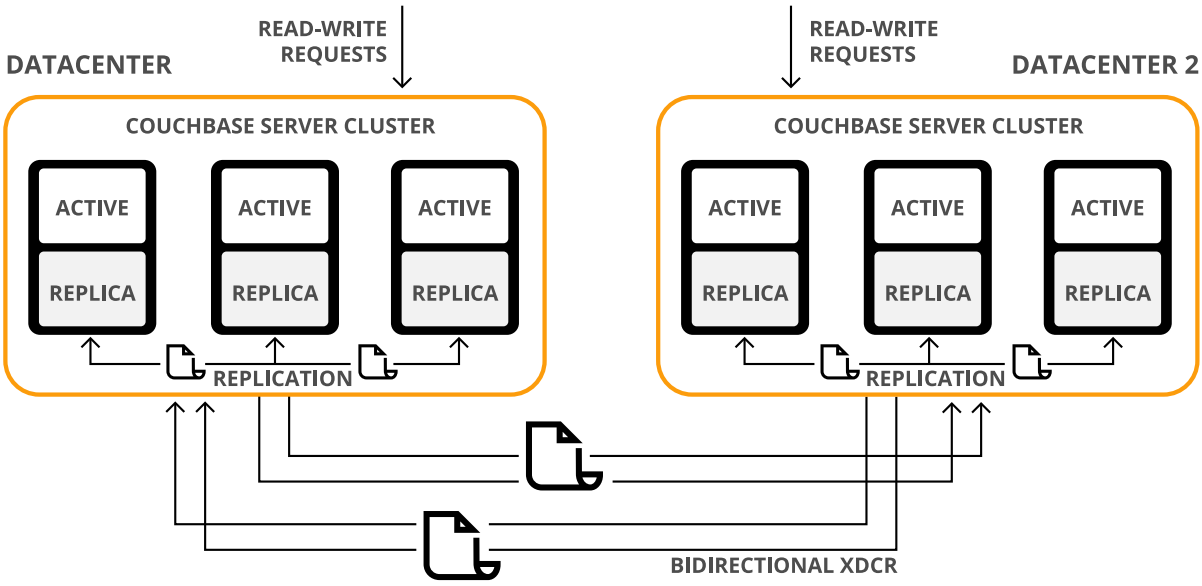
Data protection is optimal when groups are assigned equal numbers of nodes, and vBuckets are therefore distributed such that none ever occupies the same group as its associated active vBucket. By contrast, when groups are not assigned equal numbers of nodes, rebalance can only produce a best effort redistribution of replica vBuckets. This may result in replica vBuckets occupying the same group as their associated active vBuckets; meaning that data may be lost if such a group becomes unavailable.

Server group awareness considerations:

- Server group awareness only applies to the data service.
- Failover should be enabled for server groups only if three or more server groups have been established, and sufficient capacity exists to absorb the load of any failed-over group.
- The first node, and all subsequent nodes, are automatically placed in a server group named Group 1. Once you create additional server groups, you are required to specify a server group when adding additional cluster nodes.

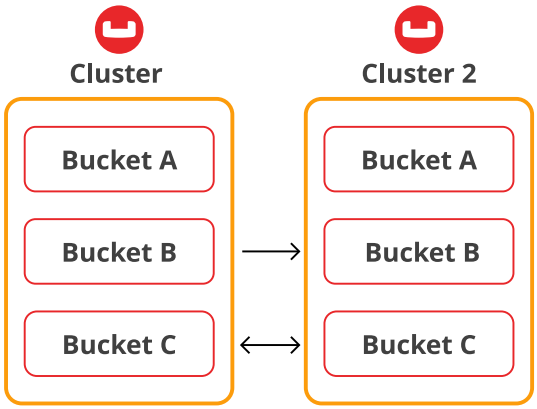
CROSS-DATACENTER REPLICATION (XDCR)

Cross-datacenter replication provides an easy way to replicate active data, while still in-memory, to multiple, geographically diverse datacenters either for disaster recovery, or availability to bring data closer to the edge and its users.



XDCR and intra-cluster replication (replication among local cluster nodes for durability) occurs simultaneously. For example, intra-cluster replication is taking place within the clusters at both Datacenter 1 and Datacenter 2, while at the same time, XDCR is replicating documents across datacenters. On each node, after a document is persisted to disk, XDCR pushes the replica documents to other clusters. On the destination cluster, replica documents received will be stored in the Couchbase-managed object cache so that replica data on the destination cluster can undergo low-latency read/write operations.

XDCR can be set up on a per-bucket or per-collection basis. Depending on your application requirements, you might want to replicate only a subset of the data in Couchbase Server between two clusters. With XDCR, you can selectively pick which buckets or collections to replicate between two clusters in a unidirectional or bidirectional fashion. Bidirectional XDCR can be set up between Bucket C on both Cluster 1 and 2. There is unidirectional XDCR between Bucket B on both clusters. Bucket A is not replicated.



XDCR provides only a single basic mechanism from which replications are built: this is the *unidirectional* replication. A *bidirectional* topology is created by implementing two *unidirectional* replications, in opposite directions, between two clusters, such that a bucket or collection on each cluster functions as both source and target.

Used in different combinations, unidirectional and bidirectional replication can support complex topologies; an example being the *ring* topology, where multiple clusters each connect to exactly two peers, so that a complete ring of connections is formed.

When a bucket is specified as the source for an XDCR replication, all data in the bucket is replicated. Thus, if replication is started between source and target buckets that initially contain different datasets, the replication process eventually establishes a complete superset of data within each bucket.

XDCR can also apply filters to replication streams within buckets and collections. This can further scope the data being delivered to its target.

XDCR supports continuous replication of data. Data mutations are replicated to the destination cluster after they are written to disk. By default, there are 32 data streams per server per XDCR connection. These streams are spread across the partitions. They move data in parallel from the source cluster to the destination cluster.

The source and destination clusters can have a different number of servers in varying topologies. If a server in the destination cluster goes down, XDCR is able to get the updated cluster topology information and continue replicating data to the available servers in the destination cluster.

XDCR is push-based. The source cluster regularly checkpoints the XDCR replication queue per partition and keeps track of what data the destination cluster last received. If the replication process is interrupted, for example, due to a server crash or intermittent network connection failure, it is not required to restart replication from the beginning. Instead, once the replication link is restored, replication can continue from the last checkpoint seen by the destination cluster.

By default, XDCR in Couchbase is designed to optimize bandwidth. This includes optimizations like mutation de-duplication as well as checking the destination cluster to see if a mutation is required to be shipped across. If the destination cluster has already seen the latest version of the data mutation, the source cluster does not send it across. However, in some use cases with active-active XDCR, you may want to skip checking whether the destination cluster has already seen the latest version of the data mutation and optimistically replicate all mutations.

Conflict Resolution

Within a cluster, Couchbase provides strong consistency at the document level. Likewise, XDCR also provides eventual consistency across clusters. Built-in conflict resolution will pick the same "winner" on both clusters if the same document was mutated on both clusters before it was replicated across. If a conflict occurs, the document with the most updates will be considered the "winner." If both the source and the destination clusters have the same number of updates for a document, additional metadata, such as numerical sequence, CAS value, document flags, and expiration TTL value are used to pick the "winner." XDCR applies the same rule across clusters to make sure document consistency is maintained.

To improve awareness and tracking, when conflicts are detected, they can be logged into controlled collections. In your replication definition, you can specify which conflict log collection or collections you want the conflict event message and the conflicting documents to be saved in. The conflict logging is informational only. The conflict is resolved using the conflict resolution mode, Sequence or Timestamp, specified in the bucket property, and replication continues. Best practice is still to design Active-Active systems that avoid conflicts.

Enhanced interoperability between XDCR and Couchbase Mobile

For large-scale, sophisticated mobile applications, organizations can create bi-directional XDCR with Mobile buckets (a bucket being used by Sync Gateway). Teams need to be using Enterprise 8.0 or higher and Couchbase Mobile 4.0 or higher.

Priority

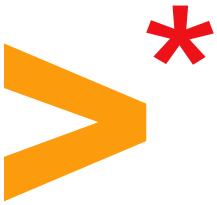
For high-throughput clusters, multiple XDCR replications may compete for resources in terms of memory and bandwidth. Different replications can be assigned a high, medium, or low priority.

Security

Couchbase Enterprise can be rendered highly secure, so as to preserve the privacy and integrity of data, and account for access attempts. Couchbase provides the following security facilities:

- **Authentication –** All administrators, users, and applications (all formally considered users) must authenticate in order to gain server access. Users can be authenticated by means of either the local or an external password registry such as LDAP or PAM. Authentication can be achieved by either passing credentials directly to the server, or by using a client certificate, in which the credentials are embedded. Connections can be secured by means of SCRAM and TLS.
- **Authorization –** Couchbase Server uses Role-Based Access Control (RBAC) to associate users with specifically assigned roles, each role corresponds to system-defined privileges, which allow degrees of access to specific system resources. During authentication, user roles are determined; therefore, authorization is granted if the role has approved system access, otherwise it is denied. LDAP Groups can be mapped to Couchbase roles which authorize users to specific database functions (i.e. read, write, update, query) on a whole cluster, a single or set of buckets, scopes within buckets, collections within scopes, and even documents within collections.
- **Auditing –** Actions performed on Couchbase Server can be audited. This allows administrators to ensure that system management tasks are being appropriately performed. Monitoring information can be collected and reported using the Couchbase integration to Prometheus, a popular cloud native monitoring tool.

Couchbase Enterprise supports the encryption of data that is at rest on disk, on the wire, and held by applications. Additionally, it provides a system of secret management, which allows essential information to the security and maintenance of Couchbase Server to be stored in encrypted form; and then decrypted and appropriately used at cluster startup.



ENCRYPTION AT REST

For Customers seeking to utilize encryption at rest, there are two options.

- Couchbase can run on top of commonly used third-party encryption tools, such as Linux Unified Key Setup (LUKS), Vormetric, Gemalto, and Protegrity, to provide a complete solution for data encryption at rest while also being accessible to all Couchbase services.
- Couchbase Enterprise provides native encryption at rest for the Data Service. This is file-level encryption using AES-256, the industry standard for strong data protection. It ensures sensitive data on disk stays secure – even if someone gets access to the physical storage. It encrypts configuration files, audit and log files, and all Data Service files – whether you're using Magma or Couchstore. Also, you get flexible encryption key options too – like AWS KMS, KMIP-based key management, and even per-bucket keys, which are great for multi-tenant setups. Since it's built in, it's easier to manage, helps with compliance, and saves you the cost of extra encryption tools. Note that other Services will also be supported soon. Speak to the Couchbase team for the latest.



Couchbase SDKs provide methods for encrypting portions of documents for added security at the application layer. Encrypted data sent back to the server is then always stored in that encrypted state, at rest, and is inaccessible without the key used by the original application layer. Couchbase cannot index or query application-encrypted fields.

Encryption over-the-wire

For an application to communicate securely with Couchbase Server, SSL/TLS must be enabled on the client side. When a TLS connection is established between a client application and Couchbase Server (running on port 18091), a handshake occurs, as defined by the TLS Handshake Protocol. As part of this exchange, the client must send to the server a cipher-suite list; which indicates the cipher-suites that the client supports, in order of preference. The server replies with a notification of the cipher-suite it has duly selected from the list. Additionally, symmetric keys to be used by client and server are selected by means of the RSA key-exchange algorithm.

Couchbase SDKs support SSL/TLS encryption and must use the Couchbase network port 11207 for secure communication. Couchbase Server uses ciphers that are accepted by default by OpenSSL.The default behavior employs high-security ciphers, built into OpenSSL, but can be overridden depending on the security level required by the cluster. Couchbase also supports self-signing of TLS certificates, and integrates with popular key and secret management services.



Moving data between nodes

Couchbase Server replicates data across a cluster to ensure high availability of data. When encrypting documents, replica copies are duly transmitted and stored in encrypted form.

Couchbase supports node-to-node transport-level security. When enabled, Couchbase will use TLS to encrypt traffic for intra-cluster communication between nodes. There are two settings: Control and All. When control-level security is enabled, only communication between the cluster manager and related services are encrypted, but data between nodes is transmitted in the clear. When all is enabled, all communication between cluster nodes is encrypted. Control level prevents attackers who have gotten on the cluster network from affecting the cluster configuration, but they could still sniff data between clusters. All prevents any unauthorized user on the network from reading any of the data over the wire.

For added security, use IPSec on the network that connects the nodes. Note that IPSec has two modes: tunnel and transport. Transport mode is recommended, as it is the easier of the two to set up, and does not require the creation of tunnels between all pairs of Couchbase nodes.

Enabling both Couchbase's node-to-node transport-level security and IPSec security is not recommended. This form of double encryption does not increase the amount of security and will dramatically affect performance and make the cluster less resilient to denial of service attacks.

Moving data between datacenters

To protect data transmitted between datacenters, TLS is used to encrypt XDCR connections. When TLS in XDCR is enabled, Couchbase uses TLS certificates. TLS versions 1.0 to 1.3 are supported. All traffic between source and destination datacenters is encrypted.

Closing

In conclusion, Couchbase Enterprise stands as a powerful, modern solution for the complexities of today's data-driven landscape. By integrating a flexible JSON foundation with familiar relational and multi-model access services, Couchbase provides a unified platform capable of supporting the most demanding operational and AI applications. Whether deployed as the fully-managed Couchbase Capella, through cloud-native automation, or as a self-managed installation, the platform delivers elite in-memory performance, billion-scale vector search, and unwavering 24x365 availability.

Ultimately, Couchbase's "polyglot persistent" architecture resolves the historical friction between RDBMS and NoSQL systems. By consolidating key-value speed, document flexibility, and relational reliability into a single engine, it eliminates the operational overhead of managing disparate technologies. As businesses look to simplify their stacks and scale efficiently, Couchbase offers a streamlined path forward – removing the need for multiple APIs and complex data modeling across systems while providing a secure, affordable, and highly scalable source of truth.

Couchbase Enterprise helps you to:

- **Power Your Performance** – Expect peak performance from your digital experiences, even at peak demand.
- **Accelerate Your Innovation** – Get to market faster and stay one step ahead of competitors with a unified data platform.
- **Simplify Your Operations** – Cut complexity and drive visibility by consolidating your legacy infrastructure and services.
- **Control Your Costs** – Optimize your infrastructure spending with a unified database that significantly reduces your TCO.



RESOURCES

This paper describes the architecture of Couchbase Enterprise Server, but the best way to get to know the technology is to download and use it. Couchbase Enterprise is a good fit for a number of use cases, including social gaming, ad targeting, content store, high availability caching, and more.

Couchbase has a rich ecosystem of adapters that support other systems:

- **SQL integration resources** include: Tableau, CData, Knowi, Talend, and more
- **Big data integration** includes Spark, Kafka, and Elasticsearch

Several flexible deployment options are available to support different environments:

- Download Couchbase Enterprise, visit **http://www.couchbase.com/downloads**
- **Couchbase Autonomous Operator** for Kubernetes and OpenShift
- Docker Hub hosts official **Couchbase Docker** images
- Couchbase Enterprise as a Service, in **Couchbase Capella**

Training is available through in-class instructor-led and free online courses. See the course catalog at: **https://learn.couchbase.com/**.

Comprehensive Couchbase documentation is available at: **https://docs.couchbase.com/**.



Couchbase

Couchbase, the operational data platform for AI, empowers businesses to succeed by bringing data to life in new ways. Major market-leading companies rely on Couchbase for mission critical operational, analytical, mobile and AI workloads. Built to replace legacy infrastructure and fragmented data services, Couchbase empowers enterprises with a unified platform architected for performance, flexibility and global scale.

With Couchbase, organizations bring their data to life, launching game-changing customer experiences, exploring the limitless potential of AI, and seamlessly extending applications from the cloud to the edge and beyond. Couchbase's AI-ready technology and enterprise partnership model eliminate complexity and reduce total cost of ownership, enabling teams to stay agile, innovative and secure.

Couchbase believes data should never slow you down, but act as the foundation for your next breakthrough. Discover why Couchbase is trusted to help the world's biggest players scale, move fast and stay resilient, no matter what's next on their roadmap. Visit couchbase.com and follow us on LinkedIn and X.

© 2026 Couchbase. All rights reserved.



